

MUSTAPHA AMBALI

Software Developer · Nigeria

+234 814 609 2185 · hello@ambalimustapha.dev

ambalimustapha.dev · github.com/ambalimustaphadev · linkedin.com/in/mustapha-ambali



I build software end to end — Flutter on the front, Flask and PostgreSQL underneath. The projects below are what that looks like in practice: modeling school attendance as presence rather than a checkbox, keeping a wallet's balance trustworthy through a failed transfer, splitting device identity from account identity so one compromised phone doesn't take down an account. I'm currently going deeper into backend engineering — distributed systems, Docker, and Go.

FEATURED PROJECTS

Regiforte *Presence & Academy Management Platform*

FLUTTER · PYTHON · FLASK · POSTGRESQL · REST APIS · JWT · QR TECHNOLOGY

A presence and attendance platform for a technology school, with a Flutter mobile client and a Flask/PostgreSQL backend I designed and built.

The core design question was how to represent attendance honestly. A classroom check-in system treats the timetable as ground truth — miss a slot and you're marked absent — which breaks down in a school where people move through shared spaces all day. I modeled presence instead: the system records physical entry and exit at a single gate, and treats the timetable as context rather than as the source of truth for lateness or absence. That one decision removes an entire category of edge cases — partial days, early departures, multi-session schedules — before they need special-casing.

Students, lecturers, staff, and administrators share one identity model instead of four separate attendance systems, because the underlying question — who is on the premises right now — doesn't change with role. Identity resolves from a physical QR card or the in-app digital ID, with the scan source stored only as metadata, so the same person is recognized the same way no matter which credential they used.

- Designed the Flask REST API and PostgreSQL schema behind the presence engine and identity model.
- Built the authentication flow end to end: registration, OTP verification, login/logout, password reset, JWT sessions.
- Chose a tablet kiosk at the entrance over dedicated scanner hardware, weighing setup cost and maintainability against the school's actual scale.
- Working directly with school staff to translate real operating rules — one entry point, one fixed schedule per student — into the data model.

Fihone *Mobile Security Platform*

FLUTTER · PYTHON · FLASK · POSTGRESQL · REST APIS

Fihone helps users monitor and protect their devices — sign-in, device registration, activity monitoring, and location tracking, tied together by a Flask backend.

The interesting part wasn't any single endpoint, it was the data model underneath it. One account can register several devices, and each device produces its own stream of activity and location data that needs to stay attributable and secure without collapsing into one unmanageable table. I modeled devices as their own entity linked to an account, rather than fields on a user record, so a device can be monitored or revoked without touching the others. Authentication follows the same logic: a lost or stolen device shouldn't be enough to compromise the account behind it, so device identity and account identity are checked separately instead of treated as one thing. The REST API reflects that split too — device and account endpoints are scoped independently, which keeps permission checks simple as monitoring features are added.

- Designed the application architecture from the Flutter client through the Flask backend and PostgreSQL schema.
- Modeled devices as a separate entity linked to accounts, supporting multiple devices per user.
- Built authentication that checks device identity and account identity separately.
- Scoped the REST API so device and account endpoints evolve independently as features are added.

Wallet Management API *Backend REST API*

PYTHON · FLASK · POSTGRESQL · SQLALCHEMY · DOCKER

A backend built specifically to go past basic CRUD — wallets, transfers, and transactions, with correctness as the real constraint.

Once a transfer moves value between two records, the problem stops being about exposing an endpoint and becomes about making sure a transfer can never leave the system half-finished. I treat the transaction as the source of truth and derive a wallet's balance from its transaction history rather than mutating it directly, so the balance always reflects what actually happened instead of a number that can drift out of sync. Transfers run inside database transactions, so a failure partway through rolls back cleanly instead of leaving one wallet debited and the other untouched. Every input is validated at the API boundary rather than trusted from the client, and keeping users, wallets, and transactions as separate normalized entities made it straightforward to add features like transfer history later without touching the balance logic.

- Designed normalized relational models for users, wallets, and transactions.
- Built RESTful endpoints with Flask and SQLAlchemy, secured with JWT authentication.

- Wrapped transfers in database transactions to guarantee atomicity on failure.
- Containerized the app with Docker for a consistent development environment.

WorldPeek *Country Information Explorer*

NEXT.JS · TYPESCRIPT · REST APIS

WorldPeek pulls country data from a public REST API and normalizes inconsistent response shapes — missing fields, differing formats — into one consistent model before anything reaches the UI. Capitals, population, currencies, languages, time zones, and neighboring countries are presented through reusable components built to handle incomplete data gracefully, with loading states for slower requests instead of a blank screen. The layout is responsive across devices, and the information hierarchy is deliberate — the details someone looks up first are the ones that are easiest to find.

github.com/ambalimustaphadev/WorldPeek

Personal Portfolio *ambalimustapha.dev*

NEXT.JS · TYPESCRIPT · TAILWIND CSS · VERCEL · RESEND

Built on the Next.js App Router, with accessibility and responsive layouts treated as requirements rather than an afterthought — the site works the same on a phone as it does on a laptop, in light mode or dark. The contact form runs on Resend instead of a static mailto link, so messages actually reach me. I put in the SEO groundwork most portfolios skip — Open Graph metadata, structured data, and a sitemap — so the site is legible to search engines and not just to people. The domain is custom, the email is a professional address rather than a free provider, and deployment is automated end to end: push to GitHub, Vercel builds and ships it.

OPEN SOURCE & PERSONAL PROJECTS

I publish my work on GitHub as I build it, and I use personal projects rather than tutorials to explore new technologies and see how my engineering decisions hold up over time.

EXPERIENCE

Independent Software Developer

2024 — Present

Designing and building complete products end to end — Flutter clients, Flask REST APIs, PostgreSQL data models, authentication, and deployment. Each project has been a different engineering problem rather than a repeat of the last: Regiforte is about modeling presence and identity correctly, Fihone is about keeping device and account security separate, the Wallet API is about transaction integrity. Architecture and system design only really click once you've made the decision yourself and lived with what it broke — most of what's above came directly out of that: building a feature, hitting its limits in a real workflow, and refactoring the part that didn't hold up.

Digital Media Designer — Crest Royal Tech Solutions

Designed marketing materials and branding assets for clients — the same turning-a-brief-into-something-concrete skill now applied to software.

TECHNICAL SKILLS

Languages Python, Dart, JavaScript, TypeScript, HTML, CSS

Frontend Flutter, Next.js, Tailwind CSS

Backend Flask, REST API Design, SQLAlchemy, JWT Authentication, Database Design

Databases PostgreSQL, SQLite, MongoDB

DevOps Docker, Git, GitHub, Vercel

Tools Postman, Android Studio, VS Code

EDUCATION & CERTIFICATIONS

B.Sc. Mass Communication — Adekunle Ajasin University, Second Class Upper Division, Sept 2023

Flutter & iOS/Android App Development (Pediforte Coding Academy) · Flutter & Dart Bootcamp (Angela Yu) · Flutter Masterclass (Hyunwoo Koh)

LEADERSHIP

Vice President (Editorial) — Nigeria Union of Campus Journalists (NUCJ), AAUA Chapter

Coordinated editorial planning and publication across a student team — the same requirements-gathering instinct I now use with Regiforte's school staff.

Continuous Learning: I learn mostly by building — starting a project, hitting a wall, then reading documentation or revisiting a decision once I understand why it didn't hold up. Right now that's Go, Docker, and distributed systems, plus going back over older architectural decisions with what I know now.